SoftwareX 24 (2023) 101597

Contents lists available at ScienceDirect

SoftwareX

journal homepage: www.elsevier.com/locate/softx

MetaCluster: An open-source Python library for metaheuristic-based clustering problems

Nguyen Van Thieu [a], Diego Oliva [b,*], Marco Pérez-Cisneros [b]

[a] Faculty of Computer Science, PHENIKAA University, Yen Nghia, Ha Dong, Hanoi 12116, Vietnam
[b] Depto. de Ingeniería Electro-Fotónica, Universidad de Guadalajara, CUCEI, Guadalajara, Jalisco, Mexico

ARTICLE INFO

Keywords:
Metaheuristic algorithms
Clustering
Evolutionary computation
Optimization

ABSTRACT

Clustering, based on metaheuristic algorithms, is a rapidly developing field. Its goal is to use these methods to reframe clustering issues as optimization problems. In this study, we propose an open-source library named MetaCluster. This library leverages the latest metaheuristic algorithms to tackle partitional clustering challenges. MetaCluster has two primary goals: to be open-source and to provide a user-friendly interface with comprehensive documentation. This design aims to facilitate usage for individuals with varying experience levels across diverse applications and domains. The current version of MetaCluster is easy to install and deploy. It offers access to 48 available datasets, over 40 performance metrics, more than 40 evaluation measurement methods, and a wide array of 200 metaheuristic algorithms.

Metadata

Nr	Code metadata description	*Please fill in this column*
C1	Current code version	*v1.2.0*
C2	Permanent link to code/repository used for this code version	https://github.com/thieu1995/MetaCluster
C3	Permanent link to reproducible capsule	https://pypi.org/project/metacluster/1.2.0/
C4	Legal code license	*GNU General Public License (GPL)*
C5	Code versioning system used	*Git*
C6	Software code languages, tools and services used	*Python*
C7	Compilation requirements, operating environments and dependencies	*Microsoft Windows, Linux, MacOS*
C8	If available, link to developer documentation/manual	https://metacluster.readthedocs.io/en/latest/
C9	Support email for questions	nguyenthieu2102@gmail.com

1. Introduction

Clustering in data mining categorizes data instances into groups (clusters) using predefined similarity metrics, typically represented as distance measurements between data points in different clusters. Clustering aims to group similar instances and separate dissimilar ones [1]. It finds widespread use in applications such as customer segmentation, document categorization, pattern recognition, information retrieval, and image processing [2–5].

However, clustering remains a significant challenge in data mining due to its unsupervised nature and the need to determine the appropriate number of clusters [3]. The unsupervised nature deals with unlabeled data, requiring the discovery of patterns without any prior knowledge of data labels (handling data lacking predefined categories or labels).

Clustering methods are typically divided into hierarchical and partitional approaches [6]. Hierarchical clustering involves merging or splitting clusters based on similarity measures. In contrast, partitional methods like *k*-Means partition data into clusters and aim to minimize the loss function such as the sum of squared errors (SSE). While partitional methods offer simplicity and efficiency, they are challenged by issues like local optima and sensitivity to initial centroids [6]. Hence, numerous existing models and new algorithms have been proposed to address these issues. One of the proposed approaches is to leverage nature-inspired algorithms (metaheuristic algorithms) [7].

Metaheuristic Algorithms (MHAs) can be categorized into various groups based on their underlying concepts [8]. For instance, algorithms inspired by human-related ideas, or interactions can be classified as human-inspired group, those rooted in evolutionary computing principles can be categorized as evolutionary-inspired group, and so forth. We visualize these groups and highlight a selection of prominent algorithms

* Corresponding authors.
E-mail addresses: thieu.nguyenvan@phenikaa-uni.edu.vn (N. Van Thieu), diego.oliva@cucei.udg.mx (D. Oliva), marco.perez@cucei.udg.mx (M. Pérez-Cisneros).

https://doi.org/10.1016/j.softx.2023.101597
Received 26 September 2023; Received in revised form 21 November 2023; Accepted 23 November 2023
Available online 6 December 2023
2352-7110/© 2023 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

within each category in Fig. 1. Detailed information about each category and algorithms can be found in Table 3 (Appendix) and in the study [9].

By utilizing predefined fitness functions to guide the search agents toward optimal solutions, MHAs can be effectively applied to partitional clustering problems [10]. Notably, they exhibit characteristics such as reasonable processing times, resilience against local optima, robustness in handling noisy data, and the capacity to explore and discover optimal data groupings, thereby yielding high-quality clustering results [10]. Several noteworthy algorithms possessing these characteristics are employed in partitional clustering tasks such as Genetic Algorithm (GA) [11], Differential Evolution (DE) [12], Coral Reefs Optimization (CRO) [13], Particle Swarm Optimization (PSO) [14], Queuing Search Algorithm (QSA) [15], Equilibrium Optimization [16], Whale Optimization Algorithm (WOA) [17], Grey Wolf Optimization (GWO) [18], Life Choice-based Optimization (LCO) [19], Henry Gas Solubility Optimization (HGSO) [20], Sparrow Search Optimization [21], Slime Mould Algorithm (SMA) [22], Artificial Ecosystem Optimization (AEO) [23], among others.

While these algorithms prove highly valuable, challenges persist when employing MHAs for partitional clustering problems. One notable challenge is selecting an appropriate fitness function, as it directly influences the search behavior of these algorithms. Consequently, choosing the most suitable fitness function for each problem is essential [7]. The No Free Lunch theorem [9] shows that different algorithms exhibit varied strengths and weaknesses and that no single algorithm excels universally across all tasks. As a result, conducting experiments with multiple fitness functions and comparing various algorithms becomes crucial.

However, the availability of libraries specifically designed for partitional clustering problems using metaheuristic algorithms is limited. Existing libraries, such as MEALPY [9], NiaPy [24], and EvoloPy [25] contain numerous metaheuristic algorithms, but they are primarily designed for general optimization tasks, excluding clustering problems. Since clustering involves optimizing cluster centroids according to a predefined objective function, requiring specialized implementation, dedicated frameworks and libraries tailored to this purpose are required. Nevertheless, users must know about these libraries, algorithms, and the ability to construct models for specific clustering problems, which can be a significant hurdle for those with limited experience in the field.

To the best of our knowledge, we have identified only two frameworks for clustering that utilize metaheuristic algorithms: LEAC [26] and EvoCluster [27]. LEAC is implemented in C++ and incorporates 22 evolutionary-inspired algorithms for addressing partitional clustering problems. However, many practitioners prefer programming languages with more extensive libraries and packages that are easier to learn than C++. Additionally, LEAC lacks diversity in algorithm selection as it mostly uses genetic algorithm modifications for fixed and variable k-clusters. EvoCluster, on the other hand, has a wider scope and is built on top of Python language. It comprises 30 datasets, 12 assessment measures, 10 metaheuristic methods, and 5 objective functions. The EvoloPy framework's capabilities are expanded by EvoCluster, which handles clustering tasks while letting users choose their own clustering algorithms, objective functions, and evaluation metrics. However, it still has a limited number of algorithms, fitness functions, evolutionary alternatives, and evaluation techniques. Besides, for users with little experience, extending or improving EvoCluster can be difficult due to the structure of the EvoCluster library. Furthermore, EvoCluster depends on EvoloPy, a library without any development before 2021. This could cause issues for users with bugs or modification requirements.

Based on the motivations above, this paper introduces a MetaCluster library, which implements metaheuristic algorithms for addressing partitional clustering problems. We enumerate the contributions of this paper as follows:

- MetaCluster library: We are introducing the MetaCluster library, an open-source Python library designed for optimizing partitional clusterings by utilizing metaheuristic algorithms.
- Provided comprehensive features: MetaCluster provides comprehensive support, encompassing APIs, examples, documentation, and test cases.
- Provided extensive resources: MetaCluster incorporates an extensive array of resources, including 48 datasets, 40 performance metrics, 40 objective functions, and over 200 metaheuristic algorithms.
- Provided three key classes:
  - Ø MhaKMeansTuner class: Leveraging more than 200 metaheuristic algorithms, this class optimizes hyperparameters for the KMeans model, including parameters such as n_clusters, init, max_iter, algorithm, etc.
  - Ø MhaKCentersClustering class: This class employs over 200 metaheuristic algorithms to optimize the K-center weights of the KMeans model. Utilizing these weights, users can determine the labels of new data points by using the nearest distance to the cluster centers.
  - Ø MetaCluster class: This class is a multi-optimizer, multi-objective, and multi-trial version of the MhaKCentersClustering class. It can employ multiple metaheuristic algorithms with various objective

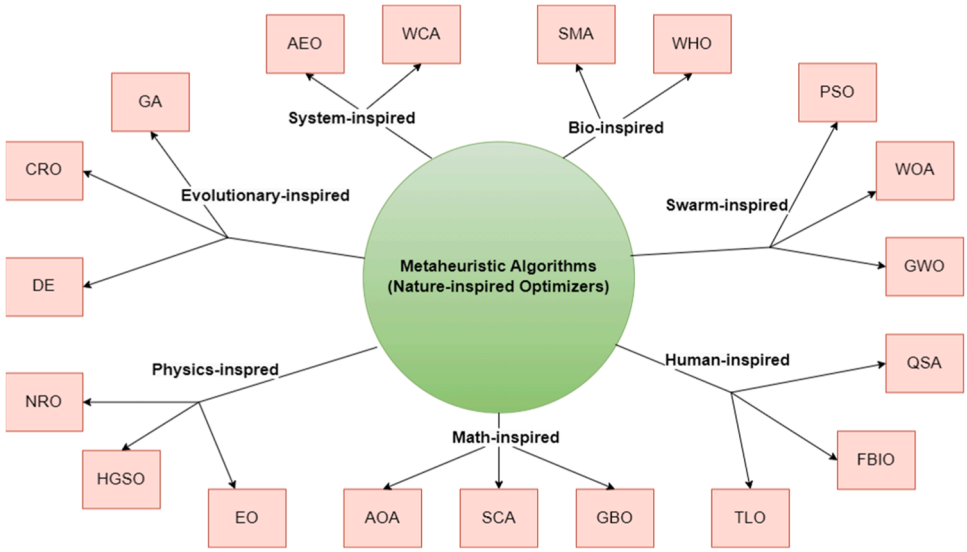

Fig. 1. The taxonomy of prominent metaheuristic algorithms.

functions and simultaneously run multiple trials to solve the K-Center Clustering problem.
- An advantage of MetaCluster is its foundation in the Python programming language, which boasts a rich ecosystem of libraries related to data mining, artificial intelligence, data science, machine learning, and deep learning.

2. Software description

2.1. Software architecture

Fig. 2 depicts the comprehensive architecture of the MetaCluster library, which comprises four primary components: the metacluster module, the mha_cluster module, the data package, and the utils package. Each of these components possesses its own classes and functions, which will be described in detail as follows:

The mha_cluster module: It is divided into two primary classes: "MhaKMeansTuner" and "MhaKCentersClustering". The "MhaKMeansTuner" class is used specifically for tuning KMeans model hyperparameters such as n_clusters, init, max_iter, and algorithm. The "MhaKCentersClustering" class, on the other hand, is used to optimize the KMeans model's K-center weights. Users can determine the labels of new data points based on these weights by calculating the closest distance to the cluster centers. Both classes can use one of the 200 available metaheuristic algorithms to optimize.

The metacluster module: It holds the most pivotal class named "**MetaCluster**". This class is a multi-optimizer, multi-objective, and multi-trial version of the **MhaKCentersClustering** class. That means it can use multiple metaheuristic algorithms with different objective functions and conduct multiple trials simultaneously to solve the K-Center Clustering issue. It also offers three methods: "**execute()**" for running the model, "**save_boxplots()**" for exporting results as boxplots, and "**save_convergences()**" for exporting convergence during the algorithm's learning phase.

The data package: It contains a set of well-known datasets suitable for clustering problems. All available datasets are labeled and can be either real-world or artificially generated. These datasets are taken from various repositories such as the scikit-learn library (SKL) [11], the UCI Machine Learning Repository (UCI) [12], the School of Computing at the University of Eastern Finland (UEF) [13], ELKI [14], and the KEEL-dataset repository (KEEL) [15].

The utils package: It provides universally shared standard components that make the framework more reuseable. The following modules and their respective uses are included in this package:

- **data_loader**: The "Data" class and auxiliary functions for reading and loading readily available data within MetaCluster are included in this module.
- **mealpy_util**: This module is essential since it contains two classes: "KMeansParametersProblem" and "KCentersClusteringProblem." The "KMeansParametersProblem" class is specifically designed to specify the hyperparameter optimization issue for the KMeans model. It will be utilized by the "MhaKMeansTuner" class. On the other hand, the "KCentersClusteringProblem" class is used to express the optimization problem of K-center weights for the KMeans model. Both of these classes derive from the Mealpy library's "Problem" class, aiming to define different optimization issues and utilize methods provided by optimizers.
- **visualize_util**: This module offers the functions to generate convergence diagrams and boxplots.
- **io_util**: This module provides operation functions for reading and writing files.
- **cluster**: Based on a wide range of performance indicators, this module's functions can help determine the ideal number of "K" cluster centers.
- **encoder**: This module contains "LabelEncoder" class, which is used to encode categorical data into numerical form.
- **validator**: This module supplies methods for checking the model parameter values against various constraints.

2.2. Software functionalities

As previously stated, one of the main goals of our library is to be extremely user-friendly and straightforward. Therefore, in this section, we will discuss the usage of the three main classes proposed by the library, which include MhaKMeansTuner, MhaKCentersClustering, and MetaCluster.

- The "MhaKMeansTuner" class comprises two main methods: fit() and predict(). In particular, the fit() function is tasked with finding the optimal set of hyperparameters for the KMeans model based on selected metaheuristic algorithms. Once the model has been configured with the best parameter set, users can invoke the predict() function responsible for assigning labels to new datasets.
- The "MhaKCentersClustering" class also includes two main methods: fit() and predict(). This class uses the chosen metaheuristic algorithm in the fit() function to learn and identify the best K-center weights for the KMeans model. After the model has been trained, users can call the predict() function, which leverages the newly acquired K-center weights to label new datasets.
- As previously stated, the "MetaCluster" class is the most important and an upgraded version of the "MhaKCentersClustering" class. As a result, it has three primary functions include execute(), save_boxplots(), and save_convergences() that correspond to the following goals:
  - Ø Models execution is the first task. Users merely need to call "**model.execute(…)**" after specifying the parameters (including the dataset, a list of algorithms, parameter lists for each method, a list of objective functions, and a list of evaluation metrics). This operation triggers all selected algorithms to train, predict the labels, and automatically save the results.
  - Ø Boxplot creation is the second task. After the models have finished running, users can simply call "**model.save_boxplots(…),**" which will save all of the boxplot visualizations for the N trials of each algorithm.
  - Ø The third task is creating convergence charts. It is feasible to provide visual representations of loss convergence during each run for each model, parameter set, and objective function. Users just need to use the "**model.save_convergences(…)**" function to save all relevant loss convergence images.

3. Experiment

3.1. Illustrative examples

a) Example of optimizing hyperparameters for the KMeans models.

This example assumes that "path-to-file" is the path to a CSV file. The data file is named "filename.csv" and has N columns representing features, with the last column being the labels for the clustering problem. Naturally, there will be no predefined labels for a clustering task. The Sum of Squared Error Index (SSEI) objective function will be utilized. The Python code snippet in Listing 1 illustrates how to use the MhaKMeansTuner class for this task. We will employ the Whale Optimization Algorithm (WOA) when invoking the fit() function, with algorithm parameters set to epoch = 10 and pop_size = 20.

a) Example of optimizing K-center weights for the KMeans model using single algorithm.

Similar to the previous example, in this scenario, we assume that the

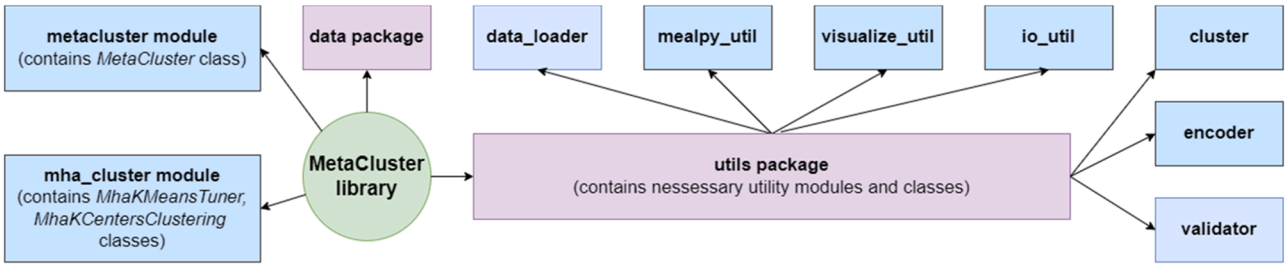

Fig. 2. The overall architecture of MetaCluster library.

```
# Import related libraries
import pandas as pd
from metacluster import Data, MhaKMeansTuner

# Set random seed for reproduce results
RANDOM_SEED = 10
# Load dataset
df = pd.read_csv("path-to-file/filename.csv")
X = df.iloc[:,0:-1].values
y = df.values[:,-1].values
# Create data object and split to train and test sets.
data = Data(X, y)
data.split_train_test(test_size=0.2, shuffle=True, inplace=True, random_state=RANDOM_SEED)
# Scale features of train and test sets
X_train, scaler = data.scale(data.X_train, method="MinMaxScaler", feature_range=(0, 1))
X_test = scaler.transform(data.X_test)

# Set up parameters for metaheuristic algorithm and define Model
paras = {"name": "WOA", "epoch": 50, "pop_size": 20}
model = MhaKMeansTuner(optimizer="OriginalWOA", optimizer_paras= paras,  seed=RANDOM_SEED)
# Train the model
model.fit(X_train, max_clusters=5, obj="SSEI", verbose=True)
# Get the best hyperparameters for KMeans model
print(model.best_parameters)
# Get the prediction labels for new dataset using the best hyperparameters
print(model.predict(X_test))
```

Listing 1. Snippet code for optimizing hyperparameters of KMeans model using MetaCluster.

path to the dataset file is "path-to-file", and the file name is "filename.csv". We will employ the hybrid GWO-KMeans model Grey Wolf Optimization (GWO)for this task. The parameters for the GWO algorithm are set to epoch = 50 and pop_size = 20. These parameters will be passed through the fit() function of the MhaKCenterClustering class, as shown in the Listing 2.

a) Example of optimizing *K*-center weights using multiple optimizers, objective functions, and trials.

In this example, we will use five hybrid models to demonstrate the capabilities of this library: FBIO-KMeans (Forensic Based Investigation Optimization), GWO-KMeans (Grey Wolf Optimization), SMA-KMeans (Slime Mould Algorithm), WOA-KMeans (Whale Optimization Algorithm), and TLO-KMeans (Teaching Learning-based Optimization). This test involved three different objective functions and five evaluation metrics. Moreover, each such algorithm was executed ten trials consecutively.

In contrast to the first two examples, in this instance, we aim to use the "circles" dataset available in the library. After loading and normalizing the dataset, we set parameters for all models, including epoch = 50 and pop_size = 30. We select a list of objective functions and a list of performance metrics to pass into the MetaCluster class. Users can set the number of runs for each case by n_trials parameters. After calling the execute() function, the models will be trained and predict labels, and then all results will be saved in five several CSV files, including:

- result_convergences.csv: This file stores fitness values during the training process.
- result_labels.csv: This file stores the labels of the dataset after training.
- result_metrics.csv: This file stores performance metrics for each run of each model.
- result_metric_mean.csv: This file stores the mean values of performance metrics (after n_trials) for each model.
- result_metric_std.csv: This file stores the standard deviation values of performance metrics (after n_trials) for each model.

As can be seen in the Listing 3, it only takes approximately 18 lines of code to accomplish everything. Increasing the number of models or adding more objective functions and performance metrics is also straightforward. Fig. 3 also illustrates boxplots and convergence charts generated during the experimental run.

```
# Import related libraries
import pandas as pd
from metacluster import Data, MhaKCentersClustering

# Set random seed for reproduce results
RD_SEED = 10
# Load dataset
df = pd.read_csv("path-to-file/filename.csv")
X = df.iloc[:,0:-1].values
y = df.values[:,-1].values
# Create data object and split to train and test sets.
data = Data(X, y)
data.split_train_test(test_size=0.2, shuffle=True, inplace=True, random_state=RD_SEED)
# Scale features of train and test sets
X_train, scaler = data.scale(data.X_train, method="StandardScaler")
X_test = scaler.transform(data.X_test)

# Set up parameters for metaheuristic algorithm and define Model
paras = {"name": "GWO", "epoch": 50, "pop_size": 20}
model = MhaKCentersClustering(optimizer="OriginalGWO", optimizer_paras=paras, seed=RD_SEED)
# Train the model
model.fit(data, cluster_finder="elbow", obj="SSEI", verbose=True)
# Get the prediction labels for new dataset using the best hyperparameters
print(model.predict(X_test))
# Get the loss during the training phase
print(model.convergence)
```

Listing 2. Snippet code for optimizing K-center weights of KMeans model using single algorithm with MetaCluster.

```
# Import related libraries
from metacluster import get_dataset, MetaCluster

# Set random seed for reproduce results
RD_SEED = 10

# Load available dataset and scaling it
data = get_dataset("circles")
data.X, scaler = data.scale(data.X, method="MinMaxScaler", feature_range=(0, 1))

# Set up list of optimizers and its parameters
list_optimizer = ["OriginalFBIO", "OriginalGWO", "OriginalSMA", "OriginalWOA", "OriginalTLO"]
list_paras = [
   {"name": "FBIO", "epoch": 50, "pop_size": 30},
   {"name": "GWO", "epoch": 50, "pop_size": 30},
   {"name": "SMA", "epoch": 50, "pop_size": 30},
   {"name": "WOA", "epoch": 50, "pop_size": 30},
   {"name": "TLO", "epoch": 50, "pop_size": 30},
]
# Set up list of objectives and performance metrics
list_obj = ["BHI", "MIS", "XBI"]
list_metric = ["BRI", "DBI", "DRI", "CHI", "KDI"]

# Define model and run it
model = MetaCluster(list_optimizer=list_optimizer, list_paras=list_paras,
                    list_obj=list_obj, n_trials=10, seed=RD_SEED)
model.execute(data=data, cluster_finder="elbow", list_metric=list_metric,
              save_path="history", verbose=False)
# Save boxplots and convergences charts
model.save_boxplots()
model.save_convergences()
```

Listing 3. Snippet code for optimizing K-center weights of KMeans model using multiple optimizers, objective functions, and trials with MetaCluster.

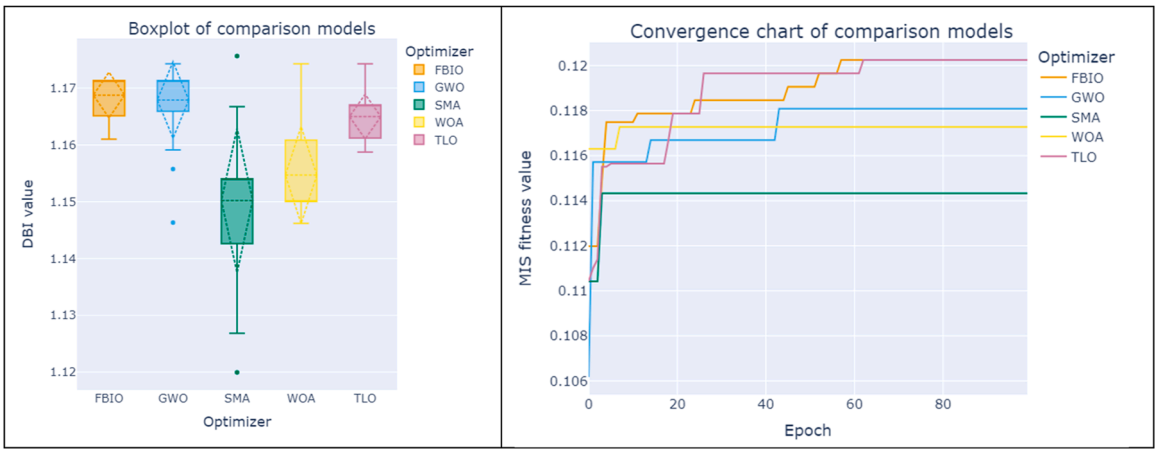

Fig. 3. The example of boxplots (left) and convergence chart (right) generated by MetaCluster.

To delve further into this library, we provide numerous additional illustrative examples at Github.[1]

3.2. Computation time comparison

In this section, we conduct a computational speed test of our proposed MetaCluster with the EvoCluster library. We design four experimental scenarios as follows:

- Scenario 1: Using 1 algorithm on 1 dataset.
- Scenario 2: Using 1 algorithm on M datasets.
- Scenario 3: Using N algorithms on 1 dataset.
- Scenario 4: Using N algorithms on M datasets.

To ensure fairness, the selected algorithms are available in both libraries: SSA, GWO, MVO, and PSO ($N = 4$). Similarly, the datasets are iris, aggregation, banknote, balance, ecoli, and smiley ($M = 6$). Table 1 contains an overview of the details of the chosen datasets.

Two key parameters influencing the algorithms are Max Generations (epoch) set to 50 and the number of search agents in the population (pop_size) set to 20. The number of test times (n_trials) is set to 3, and the chosen objective function is Sum of Squared Error (SSE).

Table 2 provides a summary of the total computation time (after n_trials runs) of scenario 1. It is evident that the proposed MetaCluster library outperforms EvoCluster across all algorithms and datasets. For example, MetaCluster's methods run around twice as fast as EvoCluster's for the Iris dataset. MetaCluster's methods run about 16 times quicker, especially for datasets like banknote with many samples. This is explained by the fact that MetaCluster uses algorithms from the Mealpy package, which is optimized for algorithm performance. Furthermore, MetaCluster's objectives and performance indicators are specifically designed and implemented to maximize speed. The remaining three scenarios exhibit the same pattern. We illustrate the computation time results for scenario 2 and scenario 3 in Fig. 4, while the computation time for scenario 4 is depicted in Fig. 5.

Table 1
The characteristics of tested datasets.

Dataset	#Samples	#Features	#Labels	Type
iris	150	5	3	Real data
aggregation	788	3	7	Artificial data
banknote	1372	5	2	Real data
balance	625	5	3	Real data
ecoli	336	8	7	Real data
smiley	500	3	4	Artificial data

4. Impact

The implemented MetaCluster is, to the best of our knowledge, the only library that implements more than 200 metaheuristic methodologies [8]. It accomplishes this as an extension of Mealpy [9], well-established within scientific research. Furthermore, MetaCluster provides extensive support for various components, including over 40 objective functions, more than 40 evaluation metrics, a wide array of available datasets (48 datasets), and multiple methods for automatically detecting the number of clusters (10 methods).

Numerous novel algorithms have recently been proposed for application in partitional clustering problems, as seen in studies [28–32]. However, the experimental sections of these papers typically do not compare these algorithms against a diverse set of state-of-the-art Metaheuristic Algorithms (MHAs). Consequently, evaluating these models' efficacy with objectivity is difficult. Therefore, the need for a library like MetaCluster, which offers many algorithms, becomes evident. This library enables researchers to swiftly construct various test scenarios, enabling unbiased comparisons with their models. Simultaneously, it speeds up the experimentation procedure.

MetaCluster presents an opportunity to introduce nature-inspired algorithms into tasks related to clustering, data mining, data science, and various applications associated with partitional clustering. In this context, MetaCluster can be a tool for comparing, testing, and evaluating existing and newly proposed algorithms.

5. Conclusions

This study introduces an open-source Python library called MetaCluster, which employs metaheuristic algorithms to address partitional clustering problems. To the best of our knowledge, MetaCluster is one of the most extensive libraries available for clustering tasks, boasting a repertoire of over 200 nature-inspired optimizers, more than 40 performance metrics, over 40 fitness functions, and an array of 48 accessible datasets. The framework is designed flexibly, ensuring developers' and researchers' ease of installation and utilization. In the future, we plan to extend the capabilities of MetaCluster by supporting parallel execution of multiple optimizers simultaneously. Additionally, we intend to incorporate more cutting-edge nature-inspired metaheuristic algorithms into the library, enhancing their utility and applicability in diverse research and practical domains.

[1] https://github.com/thieu1995/MetaCluster_examples

Table 2
Computation time of EvoCluster versus MetaCluster in scenario 1.

Dataset	EvoCluster				MetaCluster			
	SSA	GWO	MVO	PSO	SSA	GWO	MVO	PSO
iris	8.12	8.10	8.07	8.37	5.78	3.59	4.02	3.75
aggregation	33.58	32.70	33.02	34.44	6.08	4.52	5.00	4.69
banknote	83.85	77.42	78.37	77.21	5.98	4.39	4.74	4.67
balance	26.39	26.85	27.13	27.28	5.31	4.02	4.51	4.40
ecoli	14.96	15.47	15.39	14.44	5.73	4.20	6.17	4.42
smiley	20.69	22.59	21.23	21.00	5.17	3.80	4.16	4.18

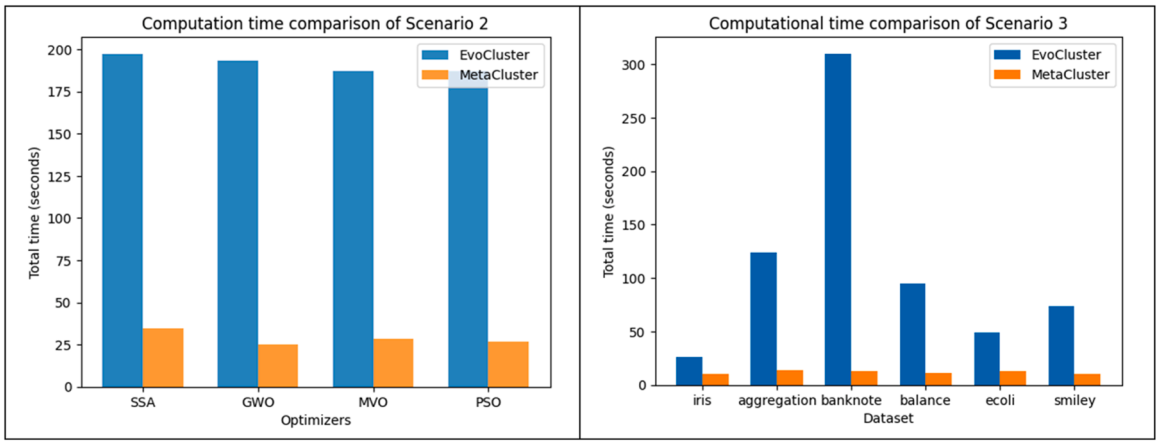

Fig. 4. The computation time of EvoCluster versus MetaCluster in scenario 2 (left) and scenario 3 (right).

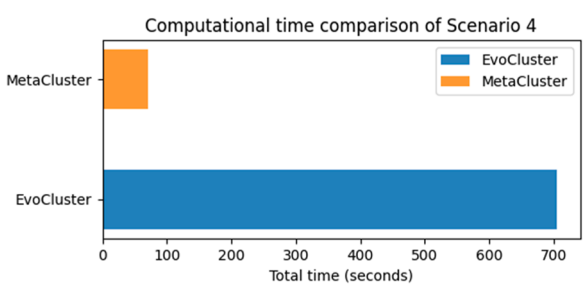

Fig. 5. The computation time of EvoCluster versus MetaCluster in scenario 4.

CRediT authorship contribution statement

Nguyen Van Thieu: Conceptualization, Investigation, Data curation, Methodology, Software, Writing – original draft, Writing – review & editing. **Diego Oliva:** Visualization, Formal analysis, Writing – original draft, Writing – review & editing, Supervision. **Marco Pérez-Cisneros:** Resources, Validation, Funding acquisition, Writing – review & editing, Project administration.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Appendix

Table 3
The taxonomy of prominent metaheuristic algorithms.

Group-inspired	Acronym	Optimizer
Evolutionary	GA	Genetic Algorithm
	DE	Differential Evolution
	CRO	Coral Reefs Optimization
Physics	NRO	Nuclear Reaction Optimization
	HGSO	Henry Gas Solubility Optimization
	EO	Equilibrium Optimization
Math	AOA	Arithmetic Optimization Algorithm
	SCA	Sine Cosine Algorithm
	GBO	Gradient-Based Optimization
Human	TLO	Teaching Learning-based Optimization
	FBIO	Forensic-Based Investigation Optimization
	QSA	Queuing Search Algorithm
Swarm	GWO	Grey Wolf Optimization

(*continued on next page*)

Table 3 (*continued*)

Group-inspired	Acronym	Optimizer
	WOA	Whale Optimization Algorithm
	PSO	Particle Swarm Optimization
Bio	SMA	Slime Mould Algorithm
	WHO	Wildebeest Herd Optimization
System	WCA	Water Cycle Algorithm
	AEO	Artificial Ecosystem-based Optimization

References

[1] Saxena A, Prasad M, Gupta A, Bharill N, Patel OP, Tiwari A, et al. A review of clustering techniques and developments. Neurocomputing 2017;267:664–81. https://doi.org/10.1016/j.neucom.2017.06.053.

[2] Kansal T, Bahuguna S, Singh V, Choudhury T. Customer segmentation using K-means clustering. In: 2018 International conference on computational techniques, electronics and mechanical systems. IEEE; 2018. p. 135–9. https://doi.org/10.1109/CTEMS.2018.8769171.

[3] Fahad A, Alshatri N, Tari Z, Alamri A, Khalil I, Zomaya AY, et al. A survey of clustering algorithms for big data: taxonomy and empirical analysis. IEEE Trans Emerg Top Comput 2014;2:267–79. https://doi.org/10.1109/TETC.2014.2330519.

[4] Afsar MM, Tayarani-N M-H. Clustering in sensor networks: a literature survey. J Netw Comput Appl 2014;46:198–226. https://doi.org/10.1016/j.jnca.2014.09.005.

[5] Murtagh F, Contreras P. Algorithms for hierarchical clustering: an overview. WIREs Data Min Knowl Discov 2012;2:86–97. https://doi.org/10.1002/widm.53.

[6] Jain AK. Data clustering: 50 years beyond K-means. Pattern Recognit Lett 2010;31:651–66. https://doi.org/10.1016/j.patrec.2009.09.011.

[7] Nanda SJ, Panda G. A survey on nature inspired metaheuristic algorithms for partitional clustering. Swarm Evol Comput 2014;16:1–18. https://doi.org/10.1016/j.swevo.2013.11.003.

[8] Molina D, Poyatos J, Del Ser J, García S, Hussain A, Herrera F. Comprehensive taxonomies of nature- and bio-inspired optimization: inspiration versus algorithmic behavior, critical analysis recommendations. Cognit Comput 2020;12:897–939. https://doi.org/10.1007/s12559-020-09730-8.

[9] Van Thieu N, Mirjalili S. MEALPY: an open-source library for latest meta-heuristic algorithms in Python. J Syst Archit 2023;139:102871. https://doi.org/10.1016/j.sysarc.2023.102871.

[10] José-García A, Gómez-Flores W. Automatic clustering using nature-inspired metaheuristics: a survey. Appl Soft Comput 2016;41:192–213. https://doi.org/10.1016/j.asoc.2015.12.001.

[11] Nguyen T, Tran N, Nguyen BM, Nguyen G. A resource usage prediction system using functional-link and genetic algorithm neural network for multivariate cloud metrics. In: 2018 IEEE 11th Conference on service-oriented computing and applications. IEEE; 2018. p. 49–56. https://doi.org/10.1109/SOCA.2018.00014.

[12] Gao W, Yen GG, Liu S. A cluster-based differential evolution with self-adaptive strategy for multimodal optimization. IEEE Trans Cybern 2014;44:1314–27. https://doi.org/10.1109/TCYB.2013.2282491.

[13] Nguyen T, Nguyen T, Nguyen BM, Nguyen G. Efficient time-series forecasting using neural network and opposition-based coral reefs optimization. Int J Comput Intell Syst 2019;12:1144. https://doi.org/10.2991/ijcis.d.190930.003.

[14] Alam S, Dobbie G, Koh YS, Riddle P, Ur Rehman S. Research on particle swarm optimization based clustering: a systematic review of literature and techniques. Swarm Evol Comput 2014;17:1–13. https://doi.org/10.1016/j.swevo.2014.02.001.

[15] Nguyen BM, Hoang B, Nguyen T, Nguyen G. nQSV-Net: a novel queuing search variant for global space search and workload modeling. J Ambient Intell Humaniz Comput 2021;12:27–46. https://doi.org/10.1007/s12652-020-02849-4.

[16] Nguyen T, Nguyen G, Nguyen BM. EO-CNN: an enhanced CNN model trained by equilibrium optimization for traffic transportation prediction. Procedia Comput Sci 2020;176:800–9. https://doi.org/10.1016/j.procs.2020.09.075.

[17] Nasiri J, Khiyabani FM. A whale optimization algorithm (WOA) approach for clustering. Cogent Math Stat 2018;5:1483565. https://doi.org/10.1080/25742558.2018.1483565.

[18] Agrawal D, Wasim Qureshi MH, Pincha P, Srivastava P, Agarwal S, Tiwari V, et al. GWO-C: grey wolf optimizer-based clustering scheme for WSNs. Int J Commun Syst 2020;33:e4344. https://doi.org/10.1002/dac.4344.

[19] Nguyen BM, Nguyen T, Vu Q-H, Tran HH, Vo H, Son DB, Binh HTT, Yu S, Wu Z. A novel nature-inspired algorithm for optimal task scheduling in fog-cloud blockchain system. IEEE Internet Things J 2023. https://doi.org/10.1109/JIOT.2023.3292872. 1–1.

[20] Ahmed AN, Van Lam T, Hung ND, Van Thieu N, Kisi O, El-Shafie A. A comprehensive comparison of recent developed meta-heuristic algorithms for streamflow time series forecasting problem. Appl Soft Comput 2021;105:107282. https://doi.org/10.1016/j.asoc.2021.107282.

[21] Nguyen T, Nguyen T, Vu Q-H, Binh Huynh TT, Minh Nguyen B. Multi-objective sparrow search optimization for task scheduling in fog-cloud-blockchain systems. In: 2021 IEEE International conference on services computing. IEEE; 2021. p. 450–5. https://doi.org/10.1109/SCC53864.2021.00065.

[22] Chen Z, Liu W. An efficient parameter adaptive support vector regression using K-means clustering and chaotic slime mould algorithm. IEEE Access 2020;8:156851–62. https://doi.org/10.1109/ACCESS.2020.3018866.

[23] Van Thieu N, Barma SD, Van Lam T, Kisi O, Mahesha A. Groundwater level modeling using augmented artificial ecosystem optimization. J Hydrol 2023;617:129034. https://doi.org/10.1016/j.jhydrol.2022.129034.

[24] Vrbančič G, Brezočnik L, Mlakar U, Fister D, Fister Jr I. NiaPy: Python microframework for building nature-inspired algorithms. J Open Source Softw 2018;3:613. https://doi.org/10.21105/joss.00613.

[25] Faris H, Aljarah I, Mirjalili S, Castillo PA, Merelo JJ. EvoloPy: an open-source nature-inspired optimization framework in Python. In: Proceedings of the 8th international joint conference on computational intelligence. SCITEPRESS - Science and Technology Publications; 2016. p. 171–7. https://doi.org/10.5220/0006048201710177.

[26] Robles-Berumen H, Zafra A, Fardoun HM, Ventura S. LEAC: an efficient library for clustering with evolutionary algorithms. Knowledge-Based Syst 2019;179:117–9. https://doi.org/10.1016/j.knosys.2019.05.008.

[27] R. Qaddoura, H. Faris, I. Aljarah, P.A. Castillo, EvoCluster: an open-source nature-inspired optimization clustering framework in Python, in: 2020: pp. 20–36. https://doi.org/10.1007/978-3-030-43722-0_2.

[28] Aljarah I, Mafarja M, Heidari AA, Faris H, Mirjalili S. Clustering analysis using a novel locality-informed grey wolf-inspired clustering approach. Knowl Inf Syst 2020;62:507–39. https://doi.org/10.1007/s10115-019-01358-x.

[29] Esmaeili H, Hakami V, Minaei Bidgoli B, Shokouhifar M. Application-specific clustering in wireless sensor networks using combined fuzzy firefly algorithm and random forest. Expert Syst Appl 2022;210:118365. https://doi.org/10.1016/j.eswa.2022.118365.

[30] Ghobaei-Arani M, Shahidinejad A. An efficient resource provisioning approach for analyzing cloud workloads: a metaheuristic-based clustering approach. J Supercomput 2021;77:711–50. https://doi.org/10.1007/s11227-020-03296-w.

[31] Kaur A, Kumar Y. A new metaheuristic algorithm based on water wave optimization for data clustering. Evol Intell 2022;15:759–83. https://doi.org/10.1007/s12065-020-00562-x.

[32] Shukri S, Faris H, Aljarah I, Mirjalili S, Abraham A. Evolutionary static and dynamic clustering algorithms based on multi-verse optimizer. Eng Appl Artif Intell 2018;72:54–66. https://doi.org/10.1016/j.engappai.2018.03.013.